

CS683: Advanced Computer Architecture

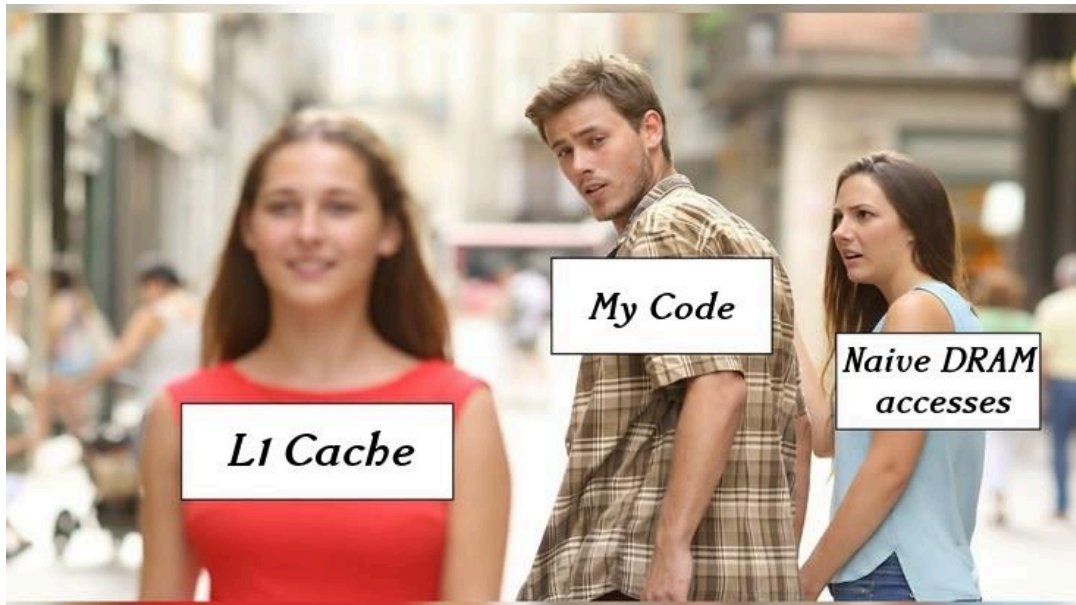
Dhurandhar Programming Assignment 1

Disclaimer: The memes in this document are intended solely for **best report award. They are not meant to offend, mislead, or be taken as factual information. Please try not to cut marks in case you do not enjoy memes.*

Team name : Cache me if you can

L1 Cache Pramod 24b1269	L2 Cache Mukunth 24b1267
L3 Cache Vishal 24b1293	DRAM Arsh 24b1266

Task 1A: Loop Transformations & Microarchitectural Exploitation



Q1) What motivated you to make changes to your code?

Ans:

Reordering:

Loop reordering was done to ensure that there is contiguous access of memory for convolutions.

Unrolling:

By unrolling loops the CPU can schedule more instructions to its execution units, thereby improving instruction parallelism, taking advantage of the processor being Out-Of-Order.

```
// for Out-Of-Order Execution
for (; ox <= W - 8; ox += 8) {
    float acc0 = 0.0f;
    float acc1 = 0.0f; // ... mapped to independent registers
```

Q2) What considerations did you take into account while implementing those changes?

Ans:

Reordering:

In reordering we had to change the loop order from the initial given order, since we knew how the arrays were stored in memory(row-major) we tried to ensure row-major access of arrays by the program. The obvious reordering is ky kx oy ox but we did not get much speed up with this, there was actually a slow down for high matrix sizes. We found that ky oy kx ox was better for larger matrices. In this order a complete row of 'ker' will go through a complete row of 'outp' reducing output matrix and input matrix memory misses , since they are highly likely to be in the L1 cache. In the obvious reordering , each element of ker will access input and output separately.

Unrolling:

As mentioned in Q1, unrolling was done to take advantage of the out-of-order parallelism, naturally the limitation here is hardware, more specifically, the number of execution units there are in our CPU. Increasing the unrolling size more than a certain amount doesn't lead to any more improvements.

```
// Initializing state for partial sums in reordered loops
for (int idx = 0; idx < H * W; ++idx) {
    output_buf[idx] = 0.0f;
}
```

Q3) How effective are your changes?(what metric will you use to quantify your effectiveness and why?)

Ans:

MPKI and **Speedup** will be used to quantify the effectiveness Speedup will be calculated on the naive implementation. Speedup's use here is quite obvious. **MPKI** is particularly useful for quantifying speedup of **Reordering**

since we claim that by switching the order we are allowing for more contiguous access of memory, we need to see if the misses go down through such access.

Metric	Naive Baseline	Obvious Reorder	Reordered	Unrolled (8x)
Speedup	1.00x	0.8x	1.47x	3.5x
<i>MPKI</i>	5.53	0.77	0.37	0.57

Task 1B: Cache Blocking & Conflict Miss Profiling

Her: He must be thinking about other women right now...

Him: How to improve the MPKI using Cache tiling??



Profiling:

L1D cache size - 40 KB (320 KB - 8 instances)
MPKI(Naive) - 0.20

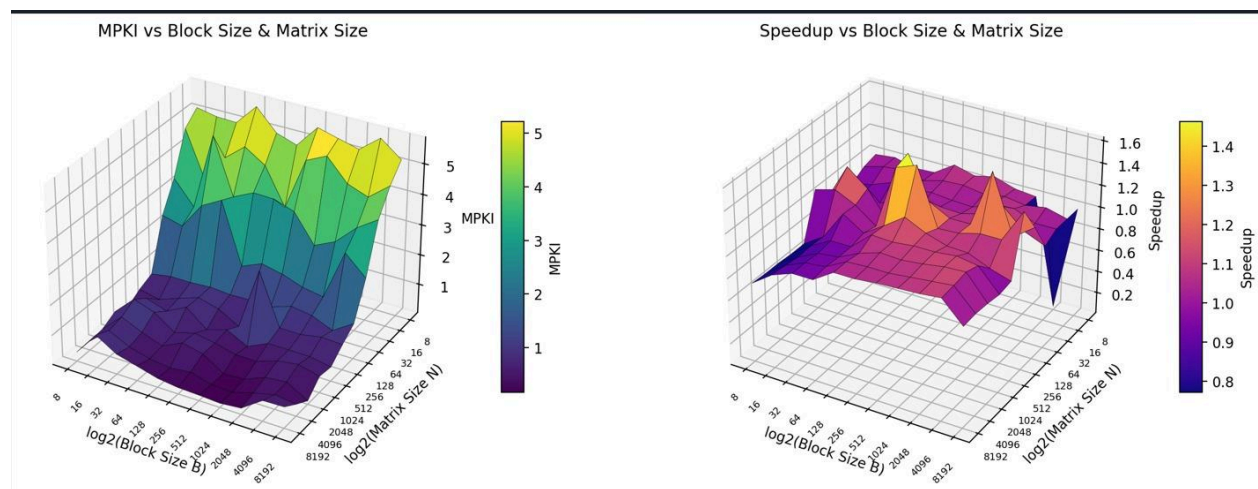
Implementing Tiled 2D Convolution

Note: The following data corresponds to a matrix size of 2048x2048 and a kernel size of 3x3.

Tile Size	MPKI	Time (ms)
8	0.37	38.746
16	0.99	35.757
32	0.68	32.362

Tile Size	MPKI	Time (ms)
64	0.45	30.857
128	0.75	29.376
256	0.6	29.189
512	0.16	28.822
1024	0.19	28.712
2048	0.48	28.279

L1D MPKI and Speedup Plots



Q1) Report the changes in **L1-D MPKI** observed when moving from naive to tiled 2D convolution. Justify your observations.

The MPKI of most of the tiled implementations is more than that of the naive implementation which at first glance looks bad. However, tile size 512 has a lesser MPKI than the naive convolution. It is to be noted however that judging by the extremely low MPKI despite **~5.9 billion instructions(atom + core)** indicates that the kernel is compute bound and not memory hungry

a.k.a the hardware prefetchers have enough time to load the data into memory before it is needed.

Q2) How did **L1-D MPKI** vary across different matrix sizes and tile sizes? Explain your findings in terms of the cache hierarchy and working set sizes.

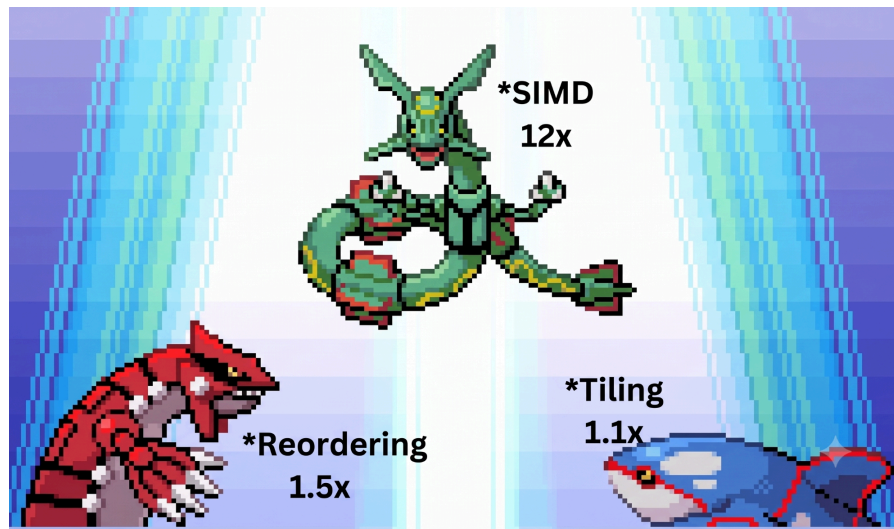
The MPKI decreases as matrix size goes up. This is because in the case of small matrices the number of instructions is very small, **MPKI** is sensitive to number of instructions so despite the low number of misses, since the matrix can be fit completely in cache, the **MPKI** remains high. As matrix size goes up, the number of instructions goes up and **MPKI** goes down.

The **Block Size** does not affect the **MPKI** much, the curve overall remains quite flat with minor changes only for each

Q3) Did you achieve a **speedup**? If yes, quantify the improvement and identify the contributing factors. If not, analyze the limiting factors and propose possible solutions.

Speedup was achieved for some sweet spots(peaks), but mostly it was **~1x speedup**. This I think is because of the ordering of the loops. The way it is ordered. For the naive it will have roughly n^2/L misses. With tiling of size B , it will have approximately $(B^2/L) * (n/B) * (n/B)$ misses which is n^2/L , i.e. The same as the naive one. Also there will be some additional overhead because we have to use additional $K/2$ halo of the region we are interested in, these regions overlap increasing mem accesses. The artifact of this can be seen when the block size is very small (8x8) and matrix size is more.

Task 1C: SIMD (AVX2) Vectorization



Instruction and SIMD Speedup Profiling:

Metric	Naive	SIMD 128	SIMD 256
CPU Core Instructions	4,733,208,109	1,657,903,954	1,139,402,308
Speedup	1x	4.23x	6.76x

SIMD Implementation:

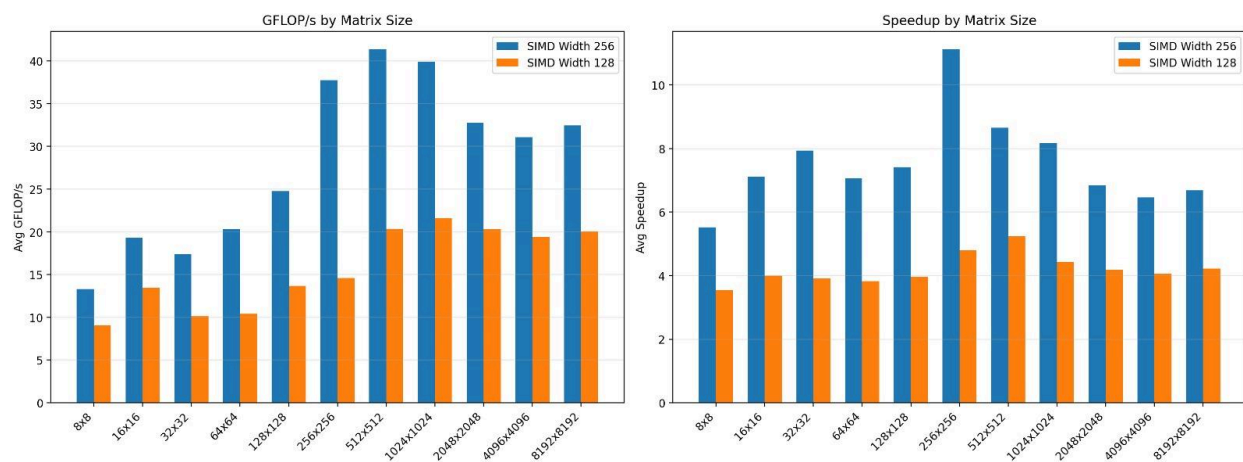
```
//256-bit(AVX-2)
for (int oy = 0; oy < H; ++oy) {
    for (int ox = 0; ox < W; ox+=8) {
        __m256 acc = _mm256_setzero_ps();
        for (int ky = 0; ky < K; ++ky) {
            for (int kx = 0; kx < K; kx++) {
                __m256 in_avx = _mm256_loadu_ps((in + (oy + ky) *
in_stride + (ox + kx)));
                __m256 ker_avx = _mm256_set1_ps(ker[ky * K +
kx]);
                acc = _mm256_fmadd_ps(in_avx, ker_avx, acc);
            }
        }
    }
}
```

```

        _mm256_storeu_ps((out + oy * W + ox), acc);
    }
}

```

Performance and Speedup Plots:



Q1) Report the change in the number of instructions you observed when moving from the naive to the SIMD implementation. Justify your observations.

The number of instructions decreased by **almost 4 times (~75% decrease)** in case of 256 bit SIMD and in case of 128 bit by almost **65% decrease** was observed. This, makes sense since SIMD 256 does calculation of **8 floats** at the same time, instead of just 1 float which should lead to an expected 8x decrease, but however we can only scale the so-called vectorizable parts of the code, we can not expect the loop to scale itself, thus we have to deal with loop overhead. We can not scale the pointer addition used to determine the next address to retrieve. Thus, **SIMD** does not scale completely.

Q2) Did you achieve any speedup? If so, how much, and what contributed to it? If not, what were the reasons?

Yes, we achieved significant speedup for SIMD when compared to other implementations, with 128-bit giving us about 4.23x and 256-bit giving us about 6.76x speedup.

The major portion of speedup was achieved by

- 1) Data Level Parallelism:** By utilizing the 256-bit wide registers multiple floating points could be stored and processed at once.
- 2) Reduced Loop Overhead:** By skipping every 8 floats(already processed) we reduce loop overhead.
- 3) Fused Add-and-Multiply:** By utilizing the fmadd, we were able to achieve fast add and multiply by specialized hardware instead of adding and multiplying separately.

Q3) Which SIMD intrinsics did you use? Justify your choice of functions.

Functions used were:

Basic: storeu, loadu

Initialization: setzero, set1

Add/Multiply: fmadd

setzero:

More specifically, the **_mm256_setzero_ps** was used. It was used to initialize a register full of 0's to initialize the accumulator.

set1:

More specifically, the **_mm256_set1_ps** was used. It was used to set all the values of the register to one value of the kernel, this value will have been slid across all the values of the **in_avx** register which contains the input registers thus, the **in_avx** must be multiplied against all of them.

fmadd:

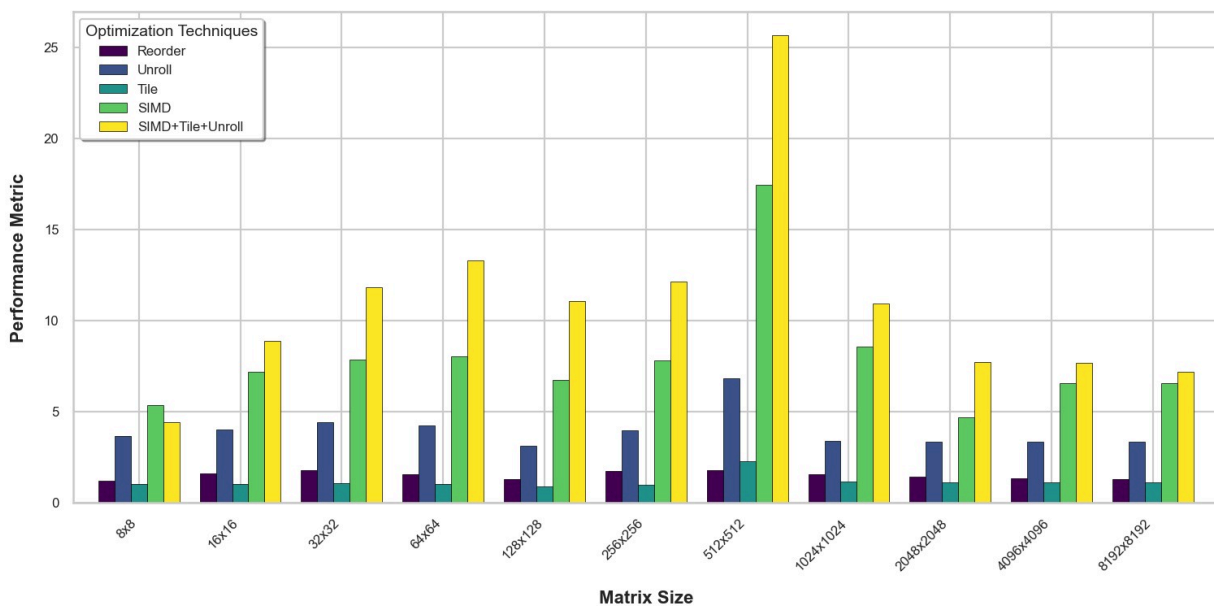
More specifically, **_mm256_fmadd_ps** was used. It was used to utilize the special multiply and add hardware available to speedup the computation further instead of using **mul** and **add** separately.

Task 1D: Sabka saath sabka vikaas



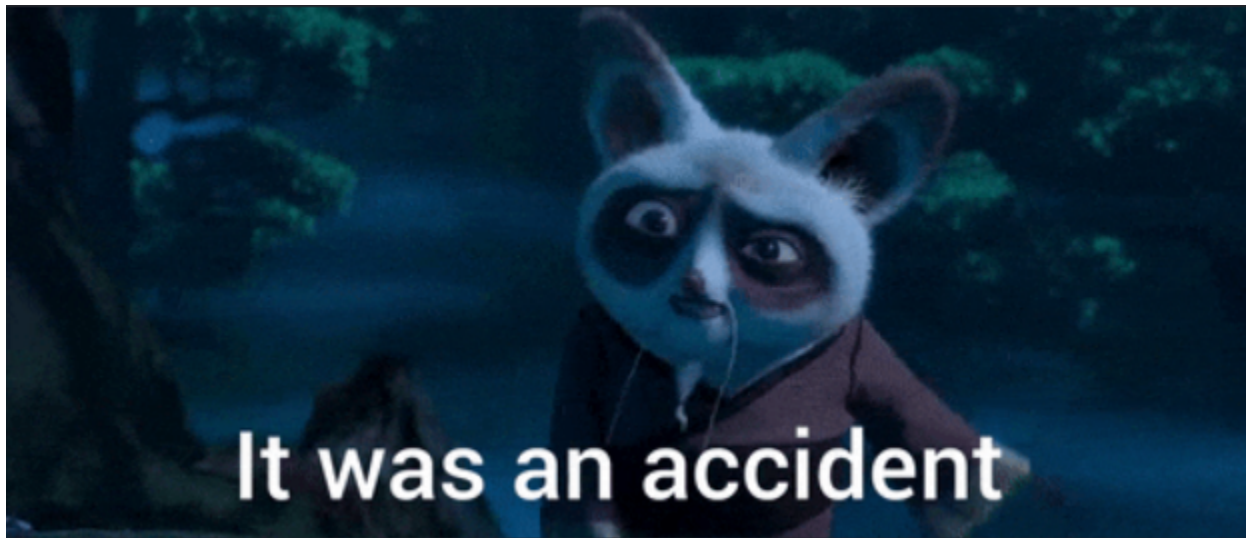
Comparsion of Different Optimization Techniques

Performance Improvement by Optimization Technique and Matrix Size



BLOOPERS

Optimised code achieving an astonishing speedup in orders of millions



```

CS683 PA-1 Task 1 2D convolution optimization harness
[smoke] all stages match the reference on small inputs.

=== Workload H=2048 W=2048 K=3 seed=1234 (GRADED) ===
stage      correct  time(ms)  GFLOP/s  speedup
-----
naive (ref)  yes      26.915    2.81     1.00x
reorder     yes      26.723    2.83     1.01x
unroll      yes      7.424     10.17    3.63x
tile        yes      14.826    5.09     1.82x
simd        yes      2.655     28.44    10.14x
optimized   yes      0.000     5807497.85 2070352.54x
-----
correctness : 40 / 40 (5 of 5 stages correct)
optimized   : 2070352.54x -> 60 / 60 speedup points
TOTAL       : 100 / 100

=== Workload H=2048 W=2048 K=5 seed=1234 (not graded) ===
stage      correct  time(ms)  GFLOP/s  speedup
-----
naive (ref)  yes      40.706    5.15     1.00x
reorder     yes      48.420    4.33     0.84x
unroll      yes      13.494    15.54    3.02x
tile        yes      45.439    4.62     0.90x
simd        yes      5.613     37.36    7.25x
optimized   yes      0.000     16131938.46 3131224.31x
-----

```

* we commented out the code ;) and ran by accident

***Arsh
Intelligence**



How Arsh feels after generating memes